SOLANA CODE ARENA

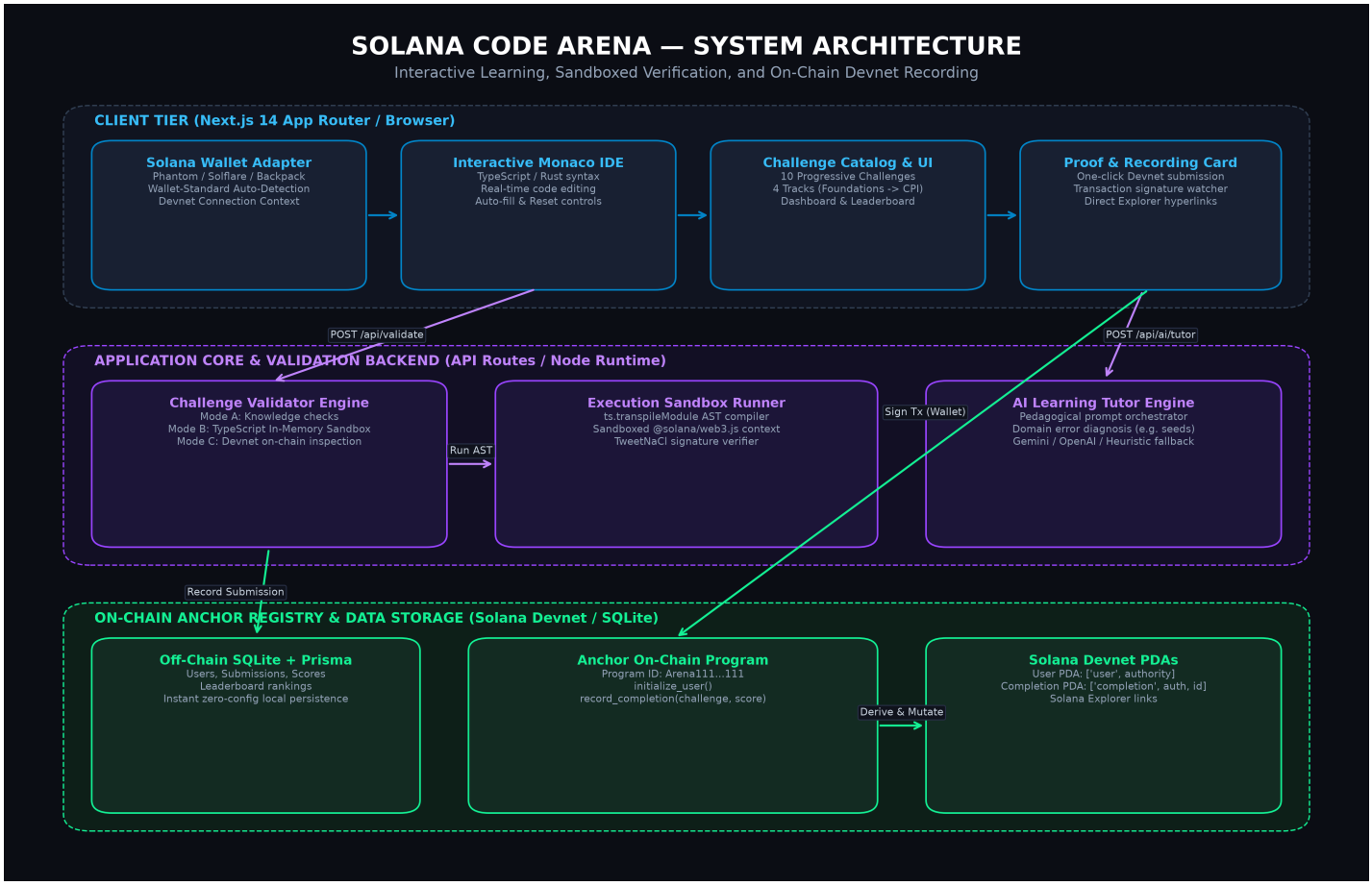
COMPLETE SYSTEM ARCHITECTURE & VERIFICATION SPECIFICATION

Core Philosophy: *"Learn Solana by building on Solana."*
A developer-first interactive learning arena pairing Monaco IDE, TypeScript sandbox verification, AI pedagogical tutoring, and immutable achievement recording on Solana Devnet.

1. Executive Architectural Overview

Solana Code Arena is designed with a strictly decoupled 4-tier architecture spanning the client IDE, the validation sandbox, persistent off-chain metadata, and an on-chain Anchor smart contract registry. The primary goal of the system is to solve the critical friction point in Web3 education: *proving that a student genuinely understood and solved a low-level Solana concept* (such as PDAs, CPI, or Token accounts) before awarding them verifiable on-chain recognition.

Figure 1: Full System Architecture Overview



The One-Sentence Architecture:
"The frontend manages the learning experience, the validator proves that a challenge was actually solved, AI helps the developer learn, and a Solana program provides an immutable record of verified progress."

2. Zero-Trust Verification Engine & Execution Sandbox

A foundational security requirement of Solana Code Arena is that **the client frontend is never trusted**. A user cannot earn a completion record or increment their leaderboard ranking simply by dispatching a network request. The verification pipeline enforces a three-stage zero-trust validation model before releasing on-chain signing capabilities:

Validation Mode	Target Area	Verification Technique
Mode A: Knowledge	Foundations, Cryptography, Conceptual rules	Deterministic schema evaluation of architectural properties and cryptographic primitives.
Mode B: Code Execution	PDAs, Instructions, Anchor, Token Math	In-memory Node sandbox with ts.transpileModule, injected @solana/web3.js, and dynamic assertion suites.
Mode C: On-Chain Inspection	Devnet Transactions, Program Deployment	Devnet RPC queries asserting account executability, program loaders, and state account byte alignments.

Figure 2: End-to-End Verification and On-Chain Recording Sequence



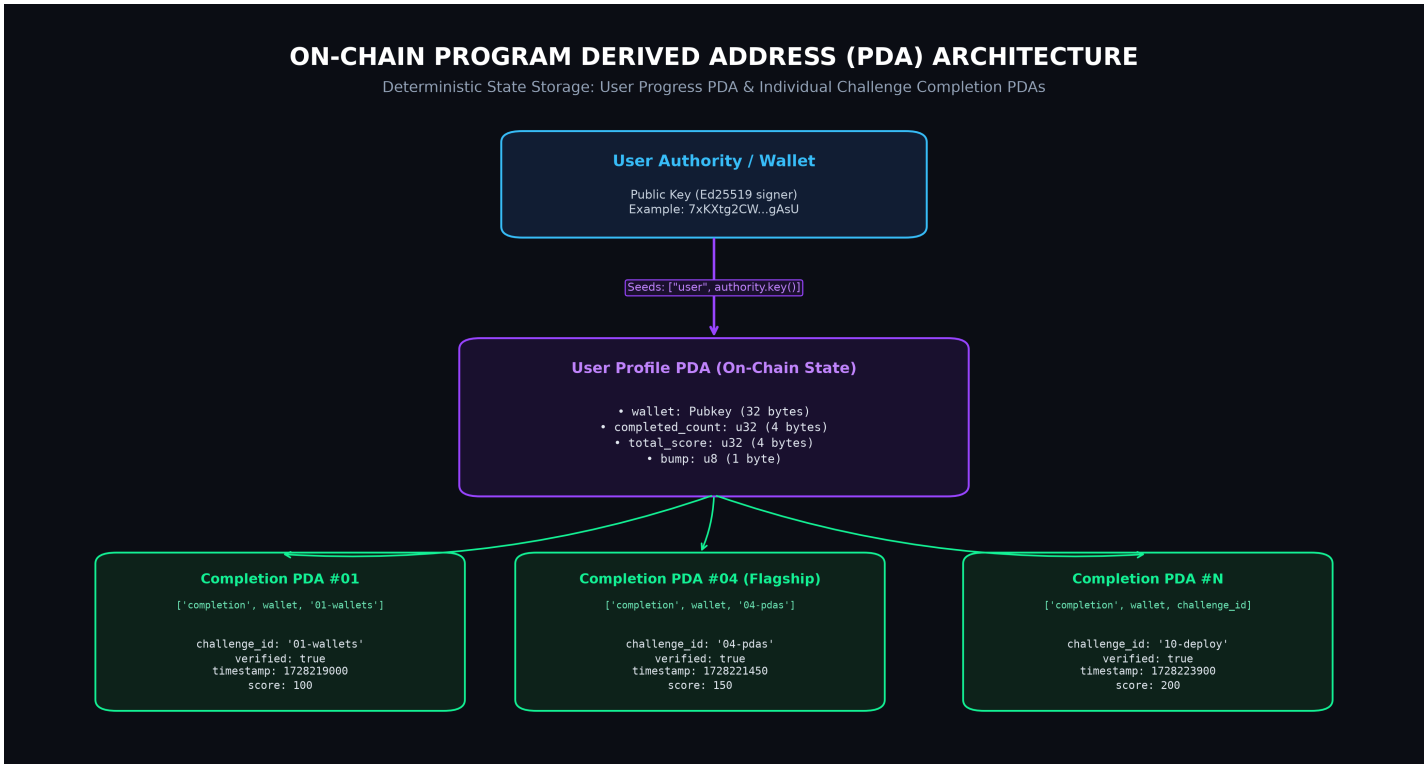
Step-by-Step Verification Lifecycle:

- **1. Monaco Code Submission:** Student writes or modifies TypeScript solution code in the browser.
- **2. Sandboxed AST Transpilation:** The validator compiles TypeScript AST to CommonJS in real-time, stripping static types while preserving runtime functions.
- **3. Assertion Execution:** Pre-defined tests execute against live `@solana/web3.js` instances (verifying off-curve properties, canonical bumps, and instruction buffers).
- **4. Proof Generation:** Upon passing all tests, the frontend unlocks the *Record on Devnet* button.
- **5. On-Chain Transaction:** The user's connected wallet (Phantom/Solflare) signs an Anchor transaction calling record_completion, initializing the Completion PDA on Devnet.

3. On-Chain Anchor Program & PDA Architecture

The on-chain Anchor program acts as an immutable, tamper-proof **achievement and verification registry**. Program Derived Addresses (PDAs) are used extensively not only for data persistence, but also to teach students how Solana eliminates global state locking by deriving deterministic, isolated account addresses off the Ed25519 curve.

Figure 3: On-Chain Program Derived Address (PDA) Hierarchy



PDA Specifications & Account Schemas:

PDA Type	Derivation Seeds	Stored State & Invariants
User Profile PDA	`["user", authority.key()]`	**wallet:** Pubkey (32B) **completed_count:** u32 (4B) **total_score:** u32 (4B) **bump:** u8 (1B) *Aggregates total builder progress.*
Completion Record PDA	`["completion", authority.key(), challenge_id.as_bytes()]`	**wallet:** Pubkey (32B) **challenge_id:** String (max 32B) **timestamp:** i64 (8B) **score:** u32 (4B) **verified:** bool (1B) **bump:** u8 (1B) *Guarantees single completion per challenge.*

Flagship Demo Day Concept (Challenge 04):

In Challenge 04, students implement deriveUserPda(wallet, programId). The validator checks that the address has no private key (off-curve), verifies that the bump seed is between 0 and 255, and asserts that the address matches the exact ground truth: PublicKey.findProgramAddressSync([Buffer.from('user'), wallet.toBuffer()], programId).

4. Pedagogical AI Coding Tutor Architecture

The AI Tutor is architected specifically as a **pedagogical learning assistant**, not a solution generator. If an AI tool simply outputs the solution, the student learns nothing. The tutor evaluates student code against the AST and provides directed, conceptual guidance.

Component	Function & Implementation
Context Assembly	Gathers challenge metadata, student code, validator error log, and specific user action (Hint, Concept, Debug Error, Explain Tx).
Pedagogical Rules	Strict system instructions prevent spoon-feeding answers. Instead, it compares expected seeds vs. actual seeds and highlights discrepancies.
Dual-Engine Pipeline	**Primary:** LLM API (Gemini 1.5 Flash / GPT-4o-mini). **Fallback:** High-speed deterministic Solana heuristics ensuring 100% offline reliability for Demo Day.

5. 10-Challenge Curriculum Specification

Track / Level	# & Title	Core Learning Objectives	Points
Level 1: Foundations	01. Wallets & Keypairs	Ed25519 cryptography, Keypair generation, message signing & detached verification	100
Level 1: Foundations	02. Transactions	Transaction building, fee payers, blockhash replay defense, SystemProgram.transfer	100
Level 1: Foundations	03. Accounts & State	Account model, lamport conversions, executable flags, program ownership	100
Level 2: Programs	**04. PDAs (Flagship)**	Off-curve addresses, seed concatenation, canonical bumps, invoke_signed privilege	150
Level 2: Programs	05. Instructions	Custom TransactionInstruction, AccountMeta signer/writable permissions, data buffer	120
Level 2: Programs	06. Anchor Framework	8-byte instruction & account discriminators derived from sha256 hashes	150
Level 3: Tokens	07. SPL Tokens	Token mints, decimal base calculations, u64 raw base unit precision	120
Level 3: Tokens	08. Token Accounts	Associated Token Accounts (ATAs), deterministic derivation from mint & wallet	130
Level 4: Advanced	09. CPI	Cross-Program Invocations, account passing, and privilege extension into System Program	180
Level 4: Advanced	10. Deploy & Verify	BPF Upgradeable Loader account structure, ProgramData accounts, Devnet verification	200

6. Technology Stack & Repository Map

Layer	Technologies / Libraries	Key Responsibilities
Frontend IDE	Next.js 14, React 18, Tailwind CSS, Monaco Editor	IDE layout, multi-track challenge browser, code editor, test runner output drawer.
Solana Integration	@solana/web3.js, Wallet-Adapter, Anchor 0.30	Wallet connection context, instruction serialization, Devnet RPC communication.
On-Chain Program	Anchor, Rust, Solana BPF runtime	User PDA management, verified completion registry, event emission.
Backend & Storage	Next.js App Router API Routes, Prisma ORM, SQLite	Challenge metadata, submission tracking, leaderboard aggregation, zero-config local DB.
Validation Sandbox	TypeScript AST Compiler, TweetNaCl	In-memory code evaluation, cryptographic signature testing, test case assertions.

7. Monorepo Directory Organization

```
fall class demo day/
■■■ apps/web/ # Next.js 14 Web Application & Developer Interface
■ ■■■ app/ # App Router: /, /challenges, /challenge/[slug], /dashboard, /leaderboard
■ ■ ■■■ api/ # API Endpoints: /api/validate, /api/completions, /api/ai/tutor, /api/leaderboard
■ ■■■ components/ # Navbar, SolanaWalletProvider, Monaco Editor integration
■ ■■■ lib/ # Internal SDK, validator, types, and challenges integration
■ ■■■ prisma/ # SQLite schema.prisma and seed script (10 challenges + leaderboard)
■■■ programs/code_arena/ # On-Chain Anchor Rust Program
■ ■■■ src/lib.rs # initialize_user, record_completion, User PDA & Completion PDA
■ ■■■ Cargo.toml # Anchor 0.30 dependencies and crate configuration
■■■ challenges/ # Master Curriculum Definitions
■ ■■■ 01-wallets/ ... 10-deploy/ # Individual challenge starter, solution, and concept docs
■ ■■■ data.ts # Master TypeScript challenge definitions and test case manifests
■■■ packages/ # Shared Monorepo Packages
■ ■■■ sdk/ # Client SDK: PDA derivation, instruction builders, Explorer links
■ ■■■ validator/ # TypeScript execution sandbox and cryptographic assertion engine
■ ■■■ types/ # Shared interfaces (Challenge, ValidationResult, UserProgress)
■■■ Anchor.toml # Anchor configuration targeting Devnet
■■■ README.md # Complete developer guide and 3-minute Demo Day script
```